ASHESI UNIVERSITY

Mobile Application Development Final Project

DataSphere: Cross-Platform Mobile Application for Campus Maintenance Reporting

Delice Ishimwe

April 26, 2026

1. Planning, Case Scenario, and Contract

a. Background Information on the Organization

The Facilities and Logistics Department at Ashesi University is responsible for maintaining the physical infrastructure of the campus. This includes managing buildings, classrooms, offices, hostels, and shared spaces to ensure they are safe, functional, and comfortable for the entire university community.

The department operates under the university's administration and is staffed by a facilities manager, maintenance technicians, and logistics support staff. They handle day-to-day maintenance tasks, respond to reported issues, procure supplies, and coordinate repairs with external contractors when necessary.

The department provides the following services to the Ashesi community:

- Electrical repairs (faulty wiring, broken outlets, lighting issues)
- Plumbing maintenance (leaks, blocked drains, broken fixtures)
- Furniture and equipment repairs (broken chairs, desks, projectors, AC units)
- Cleaning and sanitation coordination
- Emergency response for urgent campus infrastructure issues

The primary users of the department's services are Ashesi University students, academic staff, and administrative staff who live, work, and study on campus.

Currently, the Facilities and Logistics Department handles all maintenance requests through digital forms and emails. This approach makes it extremely difficult to track and respond to issues efficiently. Reports frequently get lost, go unacknowledged, or experience significant delays in resolution. There is no centralized system to log, prioritize, or update the status of maintenance requests, leaving both the reporting users and the facilities team without visibility into the progress of reported issues.

b. Overview of App Operations, Functionalities, and Capabilities

DataSphere is a cross-platform mobile application built with Flutter. It provides a centralized platform for the Ashesi University community to report, track, and manage campus facility maintenance issues. The app connects to a PHP/MySQL backend hosted on the Ashesi University server for centralized data management.

Target Users:

- Students and Staff – report maintenance issues and track their status
- Facilities Team – receive, manage, and resolve reported issues
- IT Support Team – manage user accounts and registrations via the web admin panel

Functional Requirements (FR)

FR1: Users can register and login with email and password

FR2: Students and staff can submit a maintenance report with a photo, building, room, and description

FR3: Users can take a photo using the device camera or select one from the gallery

FR4: Reports are stored locally and sync automatically when internet is restored

FR5: Users can view their report history and track status updates

FR6: Users can make a one-tap emergency call to the Facilities office

FR7: The Facilities team can view all submitted reports on a management dashboard

FR8: The Facilities team can update the status of a report (Pending, In Progress, Resolved)

FR9: The Facilities dashboard shows the most commonly reported issues on campus

FR10: Users can capture GPS coordinates of the issue location

FR11: Users can record and attach voice notes to reports

FR12: The app displays an animated splash screen on launch for five seconds

FR13: The IT Support team can create new user accounts directly from the web admin panel, assigning roles and setting credentials without requiring self-registration.

Non-Functional Requirements (NFR)

NFR1: The app should load reports and submit forms within 3 seconds on a standard mobile connection.

NFR2: The interface must be simple enough for any student or staff member to use without complexity.

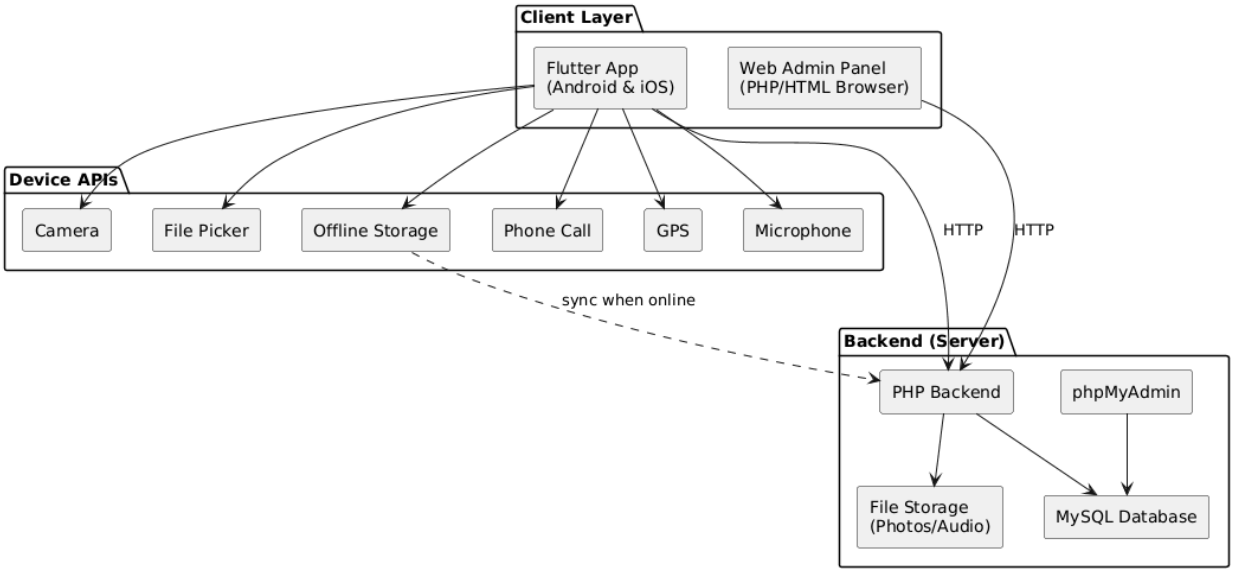
NFR3: Offline storage ensures no report is lost due to connectivity issues.

NFR4: Everyone on campus can submit reports; the Facilities dashboard is restricted to admin accounts only.

2. Prototyping, Specification, Architecture & Design

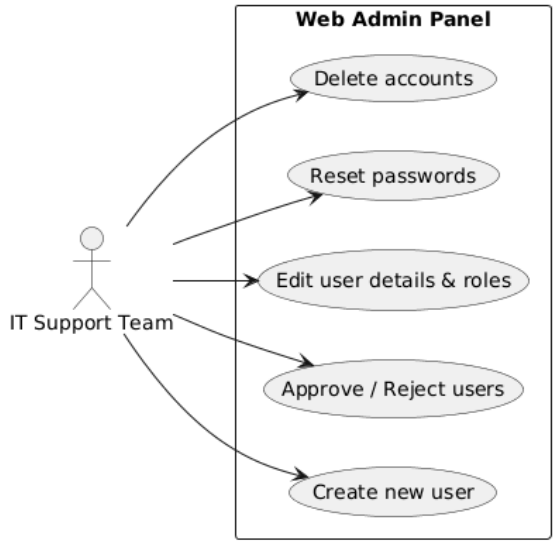
2.1 Application Architecture

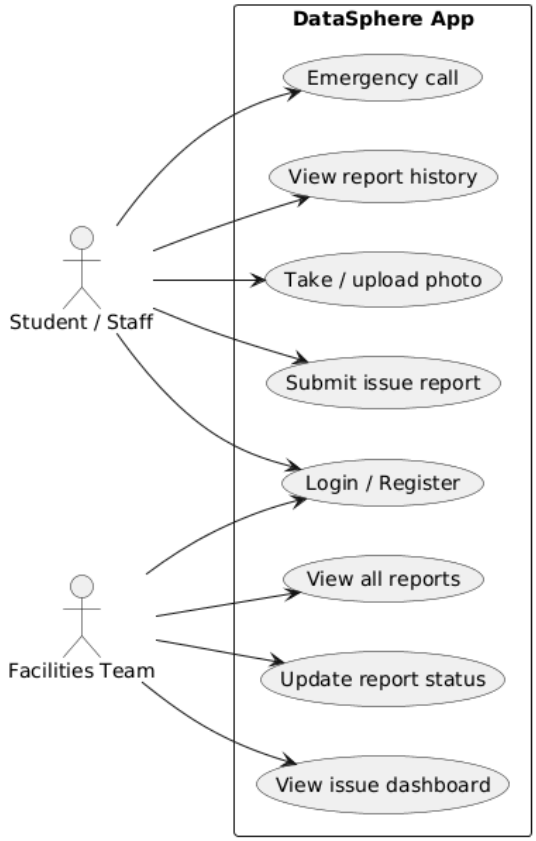
DataSphere follows a 3-layer architecture. The Flutter app runs on the user's phone and handles all screens and interactions. It connects to device features such as the camera, file picker, offline storage, GPS, and phone calls. It also communicates with a PHP backend hosted on the Ashesi server, which stores and retrieves data from a MySQL database. In addition to the mobile app, a web-based admin panel was developed using PHP and HTML, allowing the IT Support team to manage user accounts through a browser interface. Both the mobile app and the web admin panel share the same PHP/MySQL backend.



2.2 Use Case Diagram

The app has three types of users. Students and Staff can login, submit reports, upload photos, view their report history, and make emergency calls. The Facilities Team can login, view all reports, update report statuses, and view the management dashboard. The IT Support team accesses a separate web admin panel where they can approve, edit, change roles, and delete user accounts.

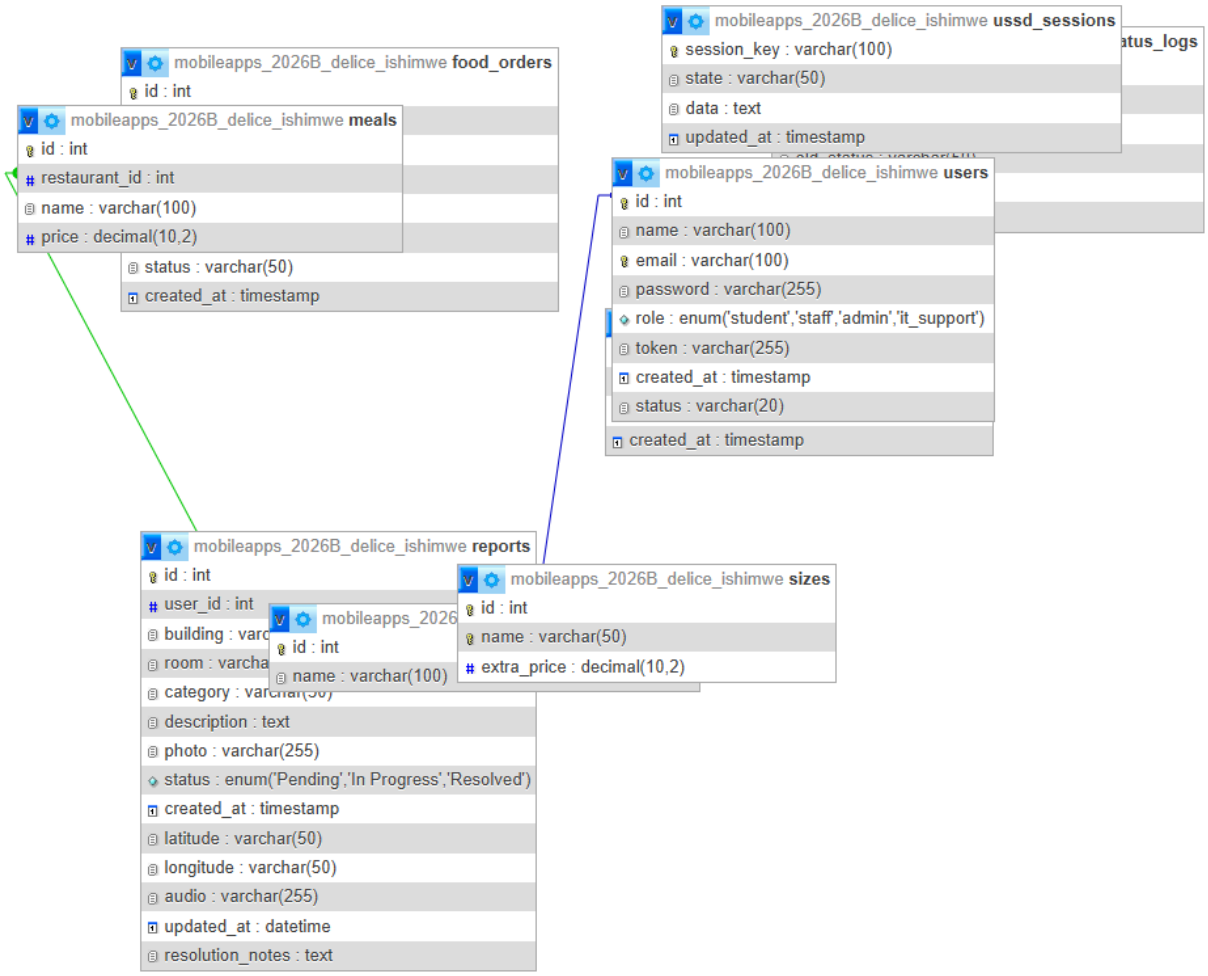




2.3 Database Design

The database has multiple tables: Users stores account details and roles, Reports stores all submitted maintenance issues, Photos stores images linked to each report, and Status_Logs tracks every status change made by the Facilities team. A status column was added to the Users table to support account approval workflow, allowing the IT admin to approve or reject new user

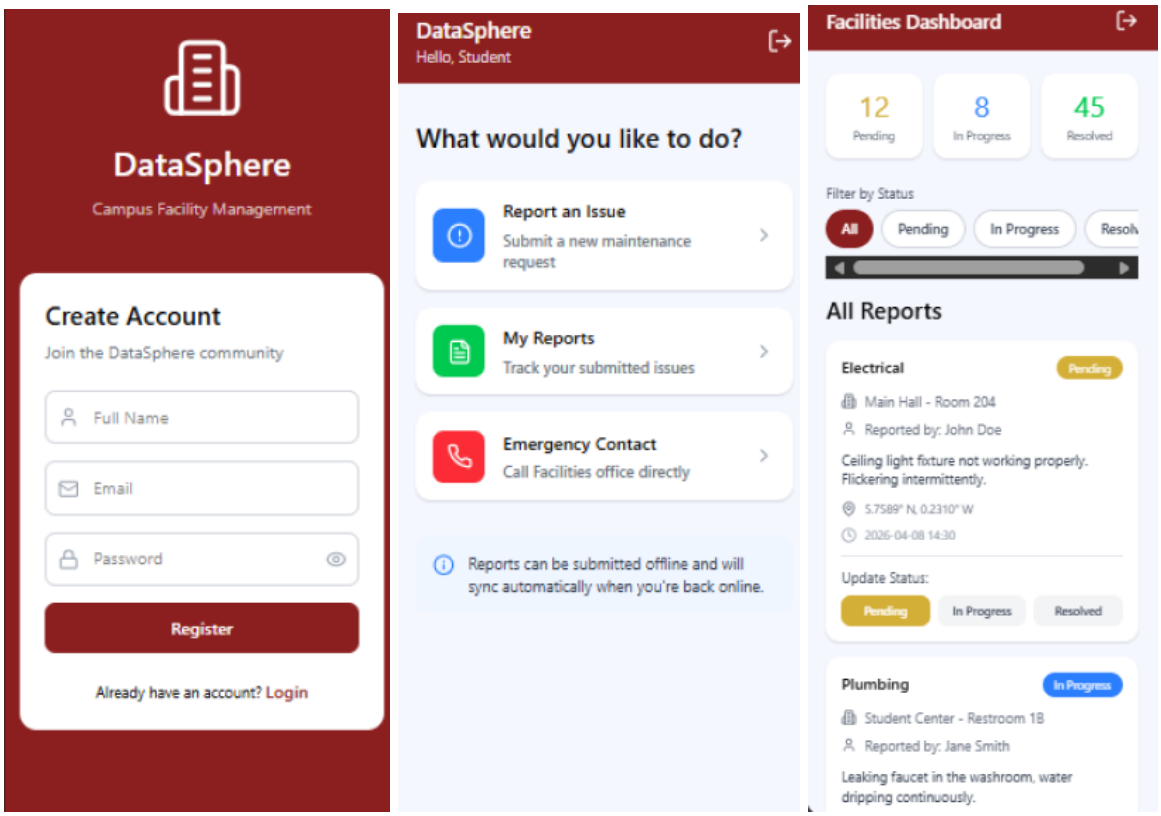
registrations. The role column was also updated to include an it_support role in addition to student, staff, and admin, separating IT administration from the Facilities team.



2.4 Prototype

A prototype of the application was designed using Figma, demonstrating the interaction between the key screens of the app.

Figure 2.4.1 Prototype screenshots



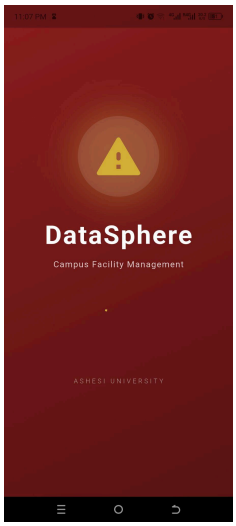
Link to the prototype:

https://www.figma.com/design/Lt3pikOFBFxbkNEO3RgE3G/Untitled?node-id=0-1&p=f

2.5 Application Screenshots

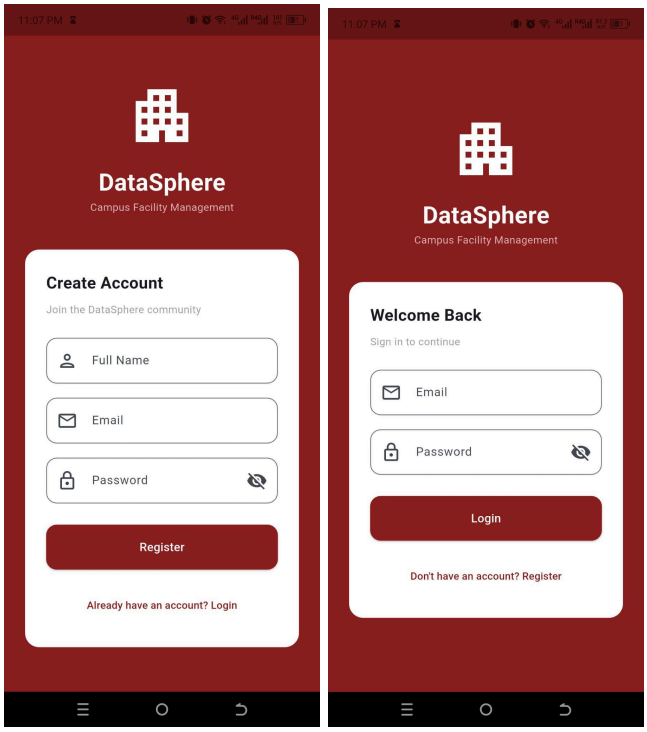
The following screenshots demonstrate the key screens and functionalities of the DataSphere application:

Figure 2.5.1: Splash Screen



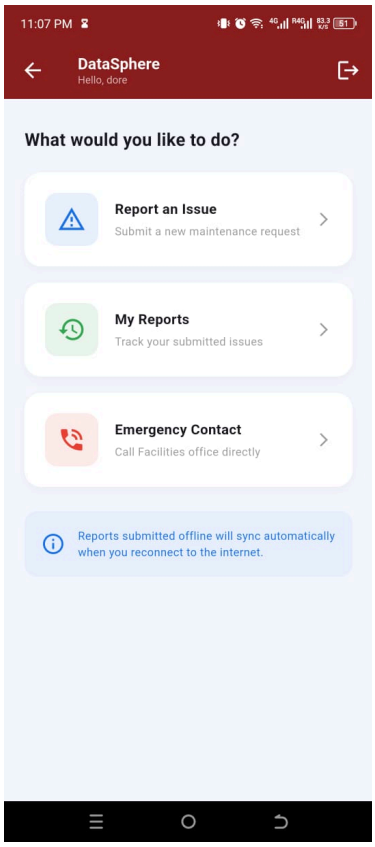
The splash screen displays the DataSphere logo and Ashesi University branding with a fade and scale animation on launch. It automatically navigates to the login screen after 5 seconds.

Figure 2.5.2: Login and Registration



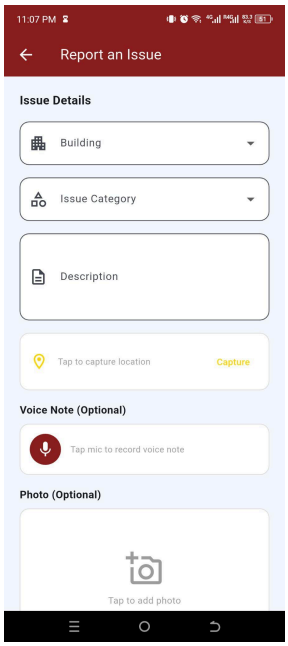
Users can create an account or login with their credentials. Passwords are securely hashed and stored in the database.

Figure 2.5.3: Student/Staff Home Screen



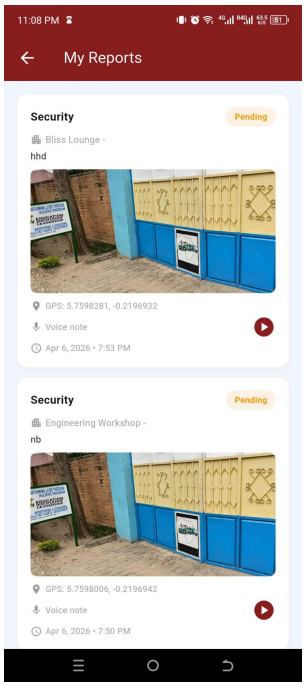
The home screen provides quick access to submit reports, view report history, and make emergency calls to the Facilities team.

Figure 2.5.4: Submit Report Screen



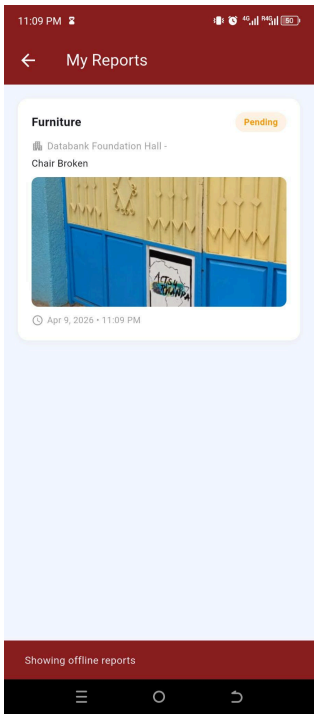
Users can select a building, issue category, add a description, capture GPS coordinates, record a voice note, and attach a photo using either the camera or gallery.

Figure 2.5.5: My Reports Screen



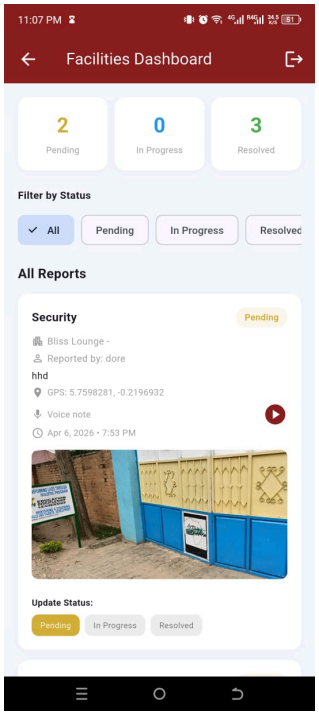
Users can view all their submitted reports with status indicators (Pending, In Progress, Resolved) and uploaded photos.

Figure 2.5.6: Offline Mode



When internet is unavailable, reports are saved locally and displayed with "You (offline)" indicator. They automatically sync when connection returns.

Figure 2.5.7: Facilities Dashboard



The Facilities team can view all reports, filter by status, update report statuses, and see reporter details.

3. IMPLEMENTATION

3.1 Development Environment and Tools

Frontend Framework:

- Flutter 3.x - Cross-platform mobile development framework for Android and iOS
- Dart programming language
- Android Studio - Integrated Development Environment (IDE)

Backend Technology:

- PHP 7.4+ - Server-side scripting language
- MySQL - Relational database management system

Local Resources Integration:

The application integrates six local resources using Flutter packages:

1. Camera Access - image_picker: ^1.0.4

- Allows users to capture photos of maintenance issues directly from the app

2. File Picker (Gallery) - image_picker: ^1.0.4

- Enables users to select existing photos from their device gallery

3. Phone Dialer - url_launcher: ^6.2.1

- Provides one-tap emergency calling to Facilities office

4. Offline Storage - sqflite: ^2.3.0

- Stores reports locally when internet is unavailable and syncs when connection returns

5. GPS/Geolocation - geolocator: ^11.0.0

- ❖ Captures precise latitude and longitude coordinates of reported issues

6. Microphone (Voice Recording) - flutter_sound: ^9.2.13 & audioplayers: ^6.0.0

- ❖ Records and plays back voice notes attached to maintenance reports

3.2 Implementation of Key Features

3.2.1 User Authentication System

The application implements secure user authentication using PHP sessions and MySQL database. When a user registers, their password is hashed using PHP's built-in password hashing function for security. User credentials are validated server-side during login, and successful authentication creates a session that is maintained locally using SharedPreferences. This allows users to remain logged in across app restarts.

3.2.2. Report Submission with Photo Upload

Students and staff can submit maintenance reports with optional photo attachments. The system uses the ImagePicker package to access both the device camera and gallery. When a photo is selected, it is uploaded to the PHP backend using HTTP MultipartRequest. Images are stored in the server's uploads directory, while the database stores only the filename reference. This approach keeps the database lightweight while maintaining access to all uploaded images.

Users can also edit or delete their pending reports. The edit screen allows them to update the building, category, and description before the Facilities team has acted on the report. Once a report is marked In Progress or Resolved, editing and deletion are disabled to protect report integrity.

3.2.3. Emergency Contact Feature

The emergency contact feature uses the url_launcher package to trigger the device's native phone dialer. When users tap the "Emergency Contact" button, the app opens the phone dialer with the Facilities team number pre-filled. The user can then tap "Call" to immediately connect. This requires the appropriate Android permissions to be declared in the AndroidManifest.xml file.

3.2.4. Offline Storage and Auto-Sync

The offline storage feature uses SQLite to ensure no reports are lost due to connectivity issues. When a user submits a report without internet, it is automatically saved to a local database table called offline_reports. These reports are displayed in the "My Reports" screen with an indicator showing they are offline. When internet connection is restored, the app automatically syncs all unsynced reports to the server in the background. Once successfully uploaded, offline reports are marked as synced in the local database.

3.2.5. Facilities Dashboard

The Facilities team dashboard displays all submitted reports from all users with real-time filtering capabilities. Staff can filter reports by status (All, Pending, In Progress, Resolved) and update report statuses using a dropdown menu. The dashboard shows reporter details, uploaded photos, and timestamps for each report. Pull-to-refresh functionality allows staff to get the latest updates instantly.

When marking a report as Resolved, the admin enters resolution notes describing how the issue was fixed. These notes are saved to the database and displayed to the reporter on their My Reports screen, providing full transparency on how their issue was handled.

3.2.6 GPS Location Capture

The application captures the precise GPS coordinates of maintenance issues using the geolocator package. When submitting a report, users can tap "Capture GPS Location" to record the exact latitude and longitude of the problem area. This feature is especially useful for outdoor issues or when the exact location within a building needs to be pinpointed. The GPS data is stored in the database and displayed on both the user's report history and the Facilities dashboard, allowing the maintenance team to navigate directly to the issue location.

3.2.7 Voice Note Recording

Users can attach voice notes to their maintenance reports using the flutter_sound package for recording and audioplayers for playback. This feature allows users to provide detailed verbal descriptions of complex issues that may be difficult to type. The voice recorder captures audio in

AAC format, stores it on the server, and enables playback directly within both the user's report history and the Facilities dashboard. This enhances report quality and gives the Facilities team better context before arriving at the site.

3.2.8 Photo Fullscreen Viewer

Users can tap any photo attached to a report to open it in a fullscreen interactive viewer. The viewer uses Flutter's InteractiveViewer widget, allowing users to pinch to zoom up to 4x and pan around the image. This is available on both the student My Reports screen and the Facilities Dashboard, making it easier to inspect the details of reported issues clearly.

3.2.9 Splash Screen

The application features a branded animated splash screen that displays when the app launches. The splash screen uses the official Ashesi University colors (maroon and gold) with smooth fade-in and scale animations. It shows for 5 seconds while checking user authentication status in the background, then automatically navigates to the appropriate screen (login, student home, or facilities dashboard). This provides a professional first impression and improves the overall user experience.

3.2.10 CRUD Operations

DataSphere implements full CRUD (Create, Read, Update, Delete) operations on maintenance reports. Authenticated users (students and staff) can Create new reports, Read and view their report history, Update pending reports through the edit screen, and Delete their own pending reports. The Facilities team can also Update report statuses with resolution notes and Delete any report from the dashboard. All operations are reflected in real-time on the PHP/MySQL backend.

3.2.11 Web Admin Panel

A web-based admin panel was built using PHP and HTML to allow the IT Support team to manage user accounts from a browser. The panel is hosted on the same Ashesi university server as the mobile app backend and connects to the same MySQL database, meaning any changes made on the web panel are immediately reflected in the mobile app.

The panel also includes a Create User form that allows the IT admin to directly register new accounts, set their role and password, and mark them as approved, which is useful for onboarding facilities team members without going through the mobile app registration flow.

The panel supports full CRUD operations on user accounts: the IT admin can view all registered users, approve or reject new registrations, edit user details such as name, email and role, reset passwords, and delete accounts. Users are protected from accidentally deleting their own account. The panel also includes search and filter functionality by role and account status.

Access to the web panel is restricted to accounts with the it_support role, which is separate from the admin role used by the Facilities team on the mobile app. This separation ensures that IT administration and facilities management are handled by different authenticated users through different interfaces.

3.3 Testing and Validation

The application was tested on physical Android devices to validate all features:

- Authentication: Successfully tested user registration, login, and session persistence
- Photo Upload: Verified both camera capture and gallery selection work correctly
- Emergency Call: Confirmed phone dialer opens with correct number pre-filled
- Offline Mode: Validated offline storage, report viewing, and automatic sync when connection returns
- Dashboard: Verified all reports display correctly and status updates work in real-time
- Web Admin Panel: Verified user approval, role changes, password reset, and deletion all reflect correctly in the mobile app in real time.

4. Appendices

4.1 Appendix C: AI Use Declaration

I used ChatGPT in two specific areas. When I was stuck on specific errors in the PHP session logic, I used it to identify what was going wrong. Also, I knew how to build the Flutter UI but used ChatGPT to speed up writing some screen sections, since my focus and time investment were on the backend logic and core app functionality.

4.2 Links

Github repo: https://github.com/Delice782/final-ind-mobile-app-proj

Video Link:
https://drive.google.com/file/d/1NMWbJXe-QrNUP6tL-vnZ78u5W9V6no7J/view?usp=sharing

APK File for DataSphere Campus Maintenance Reporting App:

https://drive.google.com/file/d/1bI0o9UbeDFoWzXkmentuLpxVj8HRwsgp/view?usp=sharing

Web Admin Panel: http://169.239.251.102:280/~delice.ishimwe/datasphere-admin/login.php

Mobile App Test Credentials

- Student/Staff:
  - You can register for a new account as you will be assigned student/staff role by default.
  - Or you can use already registered account:
    - Email:delice@gmail.com
    - Password: Delice@123
- Facilities Account:
  - Email: aku@gmail.com
  - Password: Aku@123

Web Admin Panel Credentials:

- Email: itadmin@ashesi.edu.gh
- Password: Admin1234